Tool-call gains did not survive NF4

Pernav Jain | Soup AI Engineer take-home | 10 November 2026
Run `20261110T044014005029Z` | **DON'T SHIP**

- **Dense model improves:** fully correct tool-call responses rise from 50.0% to 60.0%.
- **4-bit model loses the gain:** NF4 scores 49.3%. It loses 22 responses the dense model got right and fixes six others.
- **Release is blocked:** the general tool-call score also falls, and a preset merge-accuracy check still fails.

1. Setup and data

- **Choice:** Qwen2.5-1.5B-Instruct meets the size limit and supports an English task we can score directly: generate ordered JSON tool calls from Hermes prompts and schemas.
- **Split:** 1,600 training, 150 validation and 150 test turns. Known tool/request families stay in one split. All 1,900 rows pass structural and label-mask checks and fit within 1,024 tokens. Loss covers assistant content and end-of-turn tokens.
- **Recipe:** Soup `0.77.2`, commit `4cff43bf967e`; frozen FP16 base; LoRA rank 8, alpha 16, dropout 0.05 on seven projections per layer. AdamW, LR `1e-4`, cosine decay, 5% warmup, batch 1 and gradient accumulation 8.
- **Selection:** seeds 42/43/44 each train for two epochs, or 400 optimizer steps. Validation loss chooses each checkpoint; validation tool-call correctness chooses seed 42. Test results choose neither.
- **Hardware:** one visible T4 from Kaggle's two-card allocation; measured usable capacity 14.562 GiB. Training uses FP16 LoRA; NF4 is a separate export step.

2. Memory budget and measured peaks

The saved estimate was written before training. It records these storage floors and a **12 GiB planning budget**.

Component	Calculation	GiB
Frozen FP16 weights	1,543,714,304 x 2 bytes	2.875
LoRA + gradients + Adam states	9,232,384 x (4 + 4 + 8) bytes	0.138
Full logits, FP16 / FP32	1 x 1,024 x 151,936 x 2 / 4 bytes	0.290 / 0.580
NF4 weight-storage floor	1,310,195,712 x 0.5 + 233,373,696 x 2 bytes	1.045

Measured phase	Allocated GPU peak	Sampled device peak	Host RAM peak (RSS)
Training, maximum across seeds	4.855 GiB	5.569 GiB	3.884 GiB
Dense merge	0.034 GiB	0.138 GiB	6.436 GiB
Dense staging and NF4 save	1.089 GiB	1.247 GiB	6.132 GiB

- **Training gap:** allocation exceeds the 3.013 GiB base-plus-training-state floor by 1.842 GiB. This includes activations and loss workspaces; their separate sizes were not measured. A later calculation gives 0.082 GiB for one retained FP16 hidden state per layer, only a lower bound. The original activation allowance was coarse.
- **Available margin:** reserved GPU memory peaks at 5.430 GiB; the larger sampled device peak leaves **8.993 GiB headroom**. The 12 GiB budget was conservative. Device/RSS samples target 0.2 seconds and can miss short spikes; separate peaks cannot be added.
- **Export cost:** two dense weight copies would need 5.751 GiB of host RAM, close in scale to the measured 6.436 GiB RSS. Host buffers were not measured separately. M/Q weight files are 2.875/1.065 GiB: **2.70x compression**, limited by FP16 embeddings and quantization metadata.

3. Proof that training changed the model

- **Weights and optimizer:** all 392 LoRA tensors change, containing **9,232,384 weights**. Optimizer membership matches the trainable tensors. Seed 42 records 399 nonzero-LR steps, one zero-LR warmup step and no AMP skips. The first observed weight change is `5.01e-6`; checked frozen base matrices keep their hashes.
- **Adapter effect:** turning the adapter off matches the base's probe log probabilities exactly; turning it on changes them by up to 18.47. Coverage is four validation rows, 416 reference tokens and 16 output-distribution positions. Falling training loss, 0.7271 to 0.0023, would not prove this by itself.
- **Controls and exports:** Soup rejects the initial no-op adapter. Wrong-base and quantized-base substitutions fail our checks. M/Q reload without base/adapter-folder access; Q has 196 NF4 linear modules, nested quantization state and FP16 compute. These checks show changed, active weights and real exports; they do not prove useful behavior on every task.

4. Base, merged and 4-bit results

- **Scoring:** tool names, argument values/types, call count and order must match the reference. Invalid or length-stopped responses fail. All 150 test turns stay in the denominator. B = base; A = base + adapter; M = dense merge; Q = saved NF4. NLL is reference-token prediction loss; lower is better.

Artifact	Fully correct	Correctness	Parseable	NLL
B	75 / 150	50.00%	94.00%	0.64338
M	90 / 150	60.00%	98.00%	0.04658
Q	74 / 150	49.33%	94.67%	0.05657

- **Uncertainty:** 10,000 paired bootstrap resamples over 150 held-out groups give these 95% ranges: M-B **+10.00 pp [2.00, 18.00]**; Q-M **-10.67 pp [-17.33, -4.00]**; Q-B **-0.67 pp [-7.33, 6.00]**. Here pp means percentage points.
- **Other checks:** the bundled general tool-call suite falls from 70.0% to 57.5% to 55.0% for B/M/Q. No new critical failure appears on six fixed challenge turns; six examples cannot establish general safety.
- **Seed variation:** validation correctness is 56.67%, 54.67% and 54.67%. Only selected seed 42 receives the final test; these are not three independent test results.

5. Silent failures and what caught them

Stage	Check and observed result
Data / scorer	Strict JSON/schema checks reject wrong arguments, duplicate keys, type errors and extra/reordered calls. Soup format validation accepts our malformed-call fixture; it checks message format rather than that payload's meaning.
Tokenizer / template	Response/EOT masks pass for all 1,900 rows; separate padding/EOS checks pass. Soup doctor rejects the length-boundary fixture (exit 2), but accepts the missing-tools fixture (exit 0). Format validation accepts the double-template fixture (exit 0).
Training	Weight changes, optimizer checks, adapter on/off probes and frozen-weight hashes support actual training. Soup catches the initialized no-op adapter (exit 2).
Merge	Four sampled matrices exactly match FP16-rounded W + scaled BA. However, the largest A/M reference-log-probability gap is **0.03867**, above the preset **0.02000** limit; mean KL is 1.08e-6. This failed check needs investigation and does not, alone, prove a wrong export.
Quantization	The saved NF4 model is real, but loses **10.67 pp** against M. Correct storage settings and valid JSON do not guarantee correct calls.
Inference / release	Standalone load, identity and device checks pass. Soup `ship --evidence` rejects both M and Q using our saved measured scores. This replays its decision rules; it does not run another model evaluation.

6. Limits and changes required before release

- **Data limits:** training is 1,081/1,600 Glaive turns; test is 148/150 primary Hermes turns and is generally longer. References were not fully checked by a human for meaning or alternative valid calls. Four matrix checks, four probe rows and eight small general suites leave limited coverage.
- **DON'T SHIP:** six preset checks fail: merge fidelity, Q/B minimum gain, a positive gain interval, maximum Q/M loss, its confidence bound and general-suite retention.

1. Investigate the A/M gap using matched precision/kernels and more validation probes. Keep the failed 0.02 limit visible; do not loosen it after inspecting the result.
2. Review the 22 M-correct/Q-wrong turns, compare export changes on validation, and improve data balance/general tool-call coverage. These fixes have not yet been demonstrated.
3. Test the revised candidate on fresh held-out data. The current test has already been inspected.

Reflection and AI use

The surprise was the loss in correct calls after NF4 despite the small NLL increase. I remain concerned about reference quality and limited merge probes. I set the Soup configuration, accepted three-seed evaluation and rejected a LoRA-versus-QLoRA comparison because it was outside the task. AI helped with notebooks, checks, debugging and writing; the CPU review checks this report against saved results. I would investigate export accuracy and quantization errors before more training.

Evidence map: `docs/evidence-map.md`. Reported results and failed checks are unchanged. Full code/evidence: https://github.com/here-2007/soup-hermes-t4-takehome. Exact weights: https://www.kaggle.com/models/pernavjain/soup-experiment/PyTorch/experiment-artifacts/1.